



TADLABRICOL
| Plateforme Locale

TadlaBricol

Plateforme web de mise en relation
entre clients et artisans locaux



RAPPORT DE PROJET WEB

Réalisé par :

CHAIBI Abdellah

Encadré par :

Mme. RACHID Siham

Filière / Formation :

Développement Web & Mobile

Établissement :

ISTA Kasba Tadla

Année Universitaire : 2025 / 2026

Remerciements

Au terme de la réalisation de ce Projet de Fin d'Études, je tiens à exprimer ma profonde gratitude à ma formatrice, Mme RACHID Siham, pour son encadrement rigoureux, sa disponibilité constante ainsi que ses conseils précieux qui ont grandement contribué à la réussite de ce travail. Son soutien et ses orientations m'ont permis de surmonter plusieurs difficultés et d'acquérir de nouvelles compétences tout au long de ce projet. Je tiens également à adresser mes sincères remerciements à l'ensemble du corps professoral de l'ISTA Kasba Tadla pour la qualité de la formation dispensée, leur engagement et leurs efforts considérables dans la transmission du savoir, qui ont joué un rôle essentiel dans mon parcours académique et professionnel.

Résumé

Ce rapport présente la conception et la réalisation de **Tadla Bricole**, une plateforme web innovante dédiée à la mise en relation entre les clients et les artisans locaux. Dans un contexte où la recherche d'artisans fiables reste souvent difficile et peu structurée, ce projet vise à digitaliser et moderniser ce secteur en proposant une solution centralisée, intuitive et sécurisée. La plateforme repose sur une architecture moderne combinant un frontend développé avec React et Tailwind CSS, garantissant une interface utilisateur fluide, responsive et ergonomique, ainsi qu'un backend basé sur Laravel et MySQL, assurant une gestion efficace des données, de la sécurité et des performances. L'application intègre plusieurs fonctionnalités essentielles, notamment l'authentification sécurisée des utilisateurs, la recherche avancée d'artisans selon différents critères (localisation, spécialité, disponibilité), un système de messagerie interne permettant une communication directe, ainsi que des tableaux de bord personnalisés pour les artisans et les administrateurs. Du point de vue fonctionnel, Tadla Bricole permet aux clients de trouver rapidement des professionnels qualifiés, de consulter leurs profils, et d'interagir facilement avec eux. De leur côté, les artisans bénéficient d'un espace dédié pour gérer leurs services, améliorer leur visibilité et développer leur activité. L'administrateur dispose également d'outils de supervision pour assurer la qualité et la fiabilité de la plateforme. Ce projet répond ainsi à un besoin réel de structuration du marché artisanal local en apportant une solution numérique qui améliore la confiance, réduit les délais de recherche et optimise l'expérience utilisateur. À travers cette réalisation, l'objectif est de contribuer à la transformation digitale du secteur artisanal tout en valorisant les compétences locales.

Table des Abréviations

Abréviation	Signification
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
UI	User Interface (Interface Utilisateur)
UX	User Experience (Expérience Utilisateur)
SPA	Single Page Application
HTTP/HTTPS	HyperText Transfer Protocol (Secure)
JSON	JavaScript Object Notation
MVC	Model – View – Controller
REST	Representational State Transfer
JWT	JSON Web Token
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
i18n	Internationalization
RTL/LTR	Right-To-Left / Left-To-Right
HMR	Hot Module Replacement
IDE	Integrated Development Environment

Liste des figures

Figure 1:diagramme de Gantt de la plani 1	14
Figure 2: Schéma global de l'architecture du système.....	16
Figure 3:Diagramme de cas d'utilisation de la plateforme TadlaBricol	18
Figure 4:MCD du système TadlaBricol.....	19
Figure 5:Page d'accueil.....	35
Figure 6:Page de connexion	36
Figure 7:Page d'inscription	37
Figure 8:Section des artisans disponibles.....	38
Figure 9:Page de profil d'un artisan	39
Figure 10:Tableau de bord artisan	40
Figure 11:Page de messagerie	41
Figure 12:Tableau de bord administrateur - Vue d'ensemble.....	42
Figure 13:Tableau de bord administrateur - Gestion des utilisateurs	43
Figure 14:Tableau de bord administrateur - Gestion des artisans.....	44
Figure 15:Diagramme de séquence — Envoi d'un message.....	59
Figure 16: Diagramme de classes.....	60

Table des matières

Remerciements	2
Résumé	3
Table des Abréviations	4
Liste des figures.....	5
Introduction générale.....	9
Chapitre 1 : Contexte et Analyse du Projet	10
1.Problématique et motivations	11
2. Objectifs fonctionnels et non-fonctionnels.....	11
2.1.Objectifs fonctionnels.....	11
2.2. Objectifs non-fonctionnels	12
3. Périmètre et limites du projet	12
3.1 Périmètre couvert	12
3.2 Limites du projet	13
4. Analyse des utilisateurs	13
5.Étude comparative / Benchmark	13
6. Planification du projet	14
Chapitre 2 : Architecture et Conception du projet.....	15
1. Schéma global	16
2. Plan du site et navigation (User Flow)	16
3. Diagrammes UML utiles	17
4. Conception de la base de données	18
5. Choix techniques et justification	20
Chapitre 3 : Réalisation Technique du projet	21
1.Stack Frontend.....	22
2. Stack Backend.....	22
3.Architecture MVC :	22
4.Authentification — Laravel Sanctum.....	24
5.API REST.....	24
6.Gestion des rôles et sécurité	26
Chapitre 4 : Implémentation par Fonctionnalité	27
1.Parcours Client	28

2.Parcours Artisan	28
3.Parcours Administrateur	28
4.Messagerie.....	29
5.Internationalisation et accessibilité.....	29
6.Gestion des médias.....	29
7.Gestion des erreurs et notifications	29
Chapitre 5 : Technologies et Outils Utilisés	30
1.Technologies Frontend	31
2.Technologies Backend.....	32
3.Outils de développement	32
3.1 Environnement de Développement	32
3.2.Outils de Gestion de Projet.....	33
3.4.Outils de Test	33
3.5Outils de Gestion de Version	33
Chapitre 6 : Présentation des interfaces	34
1.Introduction.....	35
2.Page d'accueil	35
3. Page de connexion.....	36
4 Page d'inscription.....	37
5.Section des artisans disponibles	38
6. Page de profil d'un artisan.....	39
7.Tableau de bord artisan.....	40
8.Page de messagerie.....	41
9.Tableau de bord administrateur - Vue d'ensemble	42
10.Tableau de bord administrateur - Gestion des utilisateurs.....	43
11.Tableau de bord administrateur - Gestion des artisans	44
Chapitre 7 : Tests et Validation	45
1.Stratégie de test	46
2.Outils de test.....	46
2.1Tests de l'API avec Postman.....	46
2.2.Tests de l'interface utilisateur (UI)	47
3.Résultats et corrections	47
Chapitre 8 : Déploiement et Exploitation	48
1.Environnement de développement local	49
2.Plan de déploiement	49
3.Sécurité et sauvegardes envisagées	50

Chapitre 9 : Résultats, Limites et Perspectives	51
1.Résultats obtenus	52
2.Limites actuelles	52
3.Perspectives d'évolution	53
Conclusion.....	54
Bilan	55
Bibliographie.....	56
Documentation officielle :	57
Annexes	58
Diagrammes UML.....	59
Diagramme de classes	60
Configuration de l'environnement	61
Configuration frontend (.env) :	61
Extraits de code	62
Requêtes API du Backend	63
Requêtes API du Frontend.....	64

Introduction générale

Dans un contexte de transformation numérique croissante et de valorisation du patrimoine artisanal local, les plateformes de mise en relation constituent un levier puissant pour dynamiser l'économie territoriale. La région de Tadla, riche en savoir-faire artisanaux, souffre cependant d'un manque de visibilité numérique pour ses artisans.

C'est dans ce cadre que s'inscrit le projet TadlaBricol, réalisé dans le cadre d'un stage de fin de formation en Développement Web Full Stack à l'ISTA Kasba Tadla. Cette application web vise à centraliser l'offre artisanale locale, permettant aux clients de trouver facilement des artisans qualifiés, de consulter leurs profils et services, de les contacter et de partager leurs expériences via des avis.

Ce rapport présente les différentes étapes de réalisation du projet, structurées comme suit :

- Le premier chapitre présente la Commune de Kasba Tadla, cadre d'accueil du stage.
- Le deuxième chapitre décrit le contexte et l'analyse du projet de stage.
- Le troisième chapitre est consacré à l'architecture et la conception.
- Le quatrième chapitre traite de la réalisation technique.
- Les chapitres suivants présentent les fonctionnalités, les interfaces, les tests, le déploiement et les résultats obtenus.

Chapitre 1 : Contexte et Analyse du Projet

1.Problématique et motivations

Dans la région de Tadla, les artisans locaux — tels que les menuisiers, plombiers, électriciens, peintres et autres corps de métiers — exercent majoritairement leur activité dans un cadre informel. Cette réalité se caractérise par l'absence d'une présence numérique structurée, ce qui limite considérablement leur visibilité auprès d'une clientèle plus large et diversifiée. En effet, ces professionnels ne disposent généralement ni de plateformes en ligne pour présenter leurs services et leurs réalisations, ni d'outils efficaces pour interagir avec leurs clients potentiels.

Cette situation engendre plusieurs problématiques importantes, tant pour les artisans que pour les clients. Du côté des artisans, les principales difficultés résident dans l'absence d'une vitrine numérique valorisant leur savoir-faire, la difficulté d'attirer de nouveaux clients en dehors du bouche-à-oreille, ainsi que le manque d'outils leur permettant de gérer efficacement leur disponibilité, leurs prestations et leur réputation.

Du côté des clients, les contraintes sont également significatives. Trouver un artisan fiable dans leur ville reste souvent une tâche longue et incertaine. De plus, l'absence d'avis clients ou de références vérifiables rend le processus de choix encore plus complexe. Enfin, le manque de moyens de contact centralisés complique la communication et la prise de rendez-vous, ce qui peut entraîner une perte de temps et une certaine méfiance envers les prestataires.

2. Objectifs fonctionnels et non-fonctionnels

2.1.Objectifs fonctionnels

Le système proposé vise à répondre aux besoins des différents acteurs :

- Authentification sécurisée (inscription, connexion, récupération de mot de passe).
- Listing et recherche d'artisans par nom, profession, description, ville.
- Consultation des profils artisanaux avec services, photos et avis.
- Système de messagerie entre clients et artisans.
- Gestion de favoris (likes) pour sauvegarder des artisans.
- Système d'avis et de notation (1 à 5 étoiles).

- Tableau de bord artisan pour gérer profil, services, photos et disponibilité.
- Tableau de bord administrateur pour la gestion globale de la plateforme.

2.2. Objectifs non-fonctionnels

Critère	Description
Sécurité	Authentification via Laravel Sanctum, protection des routes, validation des données
Performance	SPA React avec Vite, temps de réponse rapide
Ergonomie	Interface intuitive, responsive, animations Framer Motion
Accessibilité	Support trilingue : français, anglais, arabe (RTL/LTR)
Critère	Description
Fiabilité	Stabilité du backend Laravel 12, gestion des erreurs
Évolutivité	Architecture séparée frontend/backend, API REST standardisée

3. Périmètre et limites du projet

3.1 Périmètre couvert

Le projet couvre l'ensemble du cycle de vie de la mise en relation artisanale, incluant :

- La création et gestion de comptes utilisateurs (clients, artisans, administrateurs).
- La recherche et le filtrage des artisans par profession et ville.
- La consultation des profils artisanaux avec services, photos et avis.
- Le système de messagerie entre clients et artisans.
- La gestion des favoris (likes).
- Le tableau de bord artisan pour la gestion de la présence numérique.
- L'administration et la supervision de la plateforme.
- Le support multilingue (français, anglais, arabe) avec gestion RTL/LTR.

3.2 Limites du projet

Malgré ses fonctionnalités, le système présente certaines limites :

- Absence de notifications en temps réel (WebSocket / push notifications).
- Absence d'intégration avec des systèmes de paiement en ligne.
- Performances non testées à grande échelle.
- Optimisation mobile partielle — le design responsive est en cours d'amélioration.
- URL de l'API encore codée en dur dans certains fichiers frontend.
- Token de réinitialisation de mot de passe retourné dans la réponse (mode développement uniquement).

Ces limites constituent des axes d'amélioration pour les futures versions de la plateforme.

4. Analyse des utilisateurs

Type d'utilisateur	Rôle	Besoins principaux
Client (user)	Chercher et contacter des artisans	Interface simple, recherche efficace, avis fiables
Artisan	Gérer son profil et ses services	Tableau de bord complet, gestion des photos et disponibilité
Administrateur	Superviser la plateforme	Contrôle total des utilisateurs et artisans

5. Étude comparative / Benchmark

Avant de définir les fonctionnalités de TadlaBricol, une étude comparative des solutions existantes a été réalisée :

Solution	Avantages	Limites
Réseaux sociaux (Facebook)	Gratuit, large audience	Non structuré, pas de profil professionnel dédié
Annuaire en ligne	Simple à utiliser	Pas de système d'avis, pas de messagerie
Plateformes généralistes (Jumia, etc.)	Fonctionnalités complètes	Non adapté aux artisans locaux, commission élevée
Bouche-à-oreille	Fiable, gratuit	Portée très limitée, non scalable
TadlaBricol (solution proposée)	Centralisée, adaptée, multilingue	Version initiale — améliorations progressives prévues

TadlaBricol se distingue par son adaptation au contexte local de Tadla, son interface trilingue, son système d'avis intégré et sa messagerie directe — des fonctionnalités absentes des solutions génériques existantes.

6. Planification du projet

Le développement a été structuré en plusieurs phases clés sur la durée du stage :

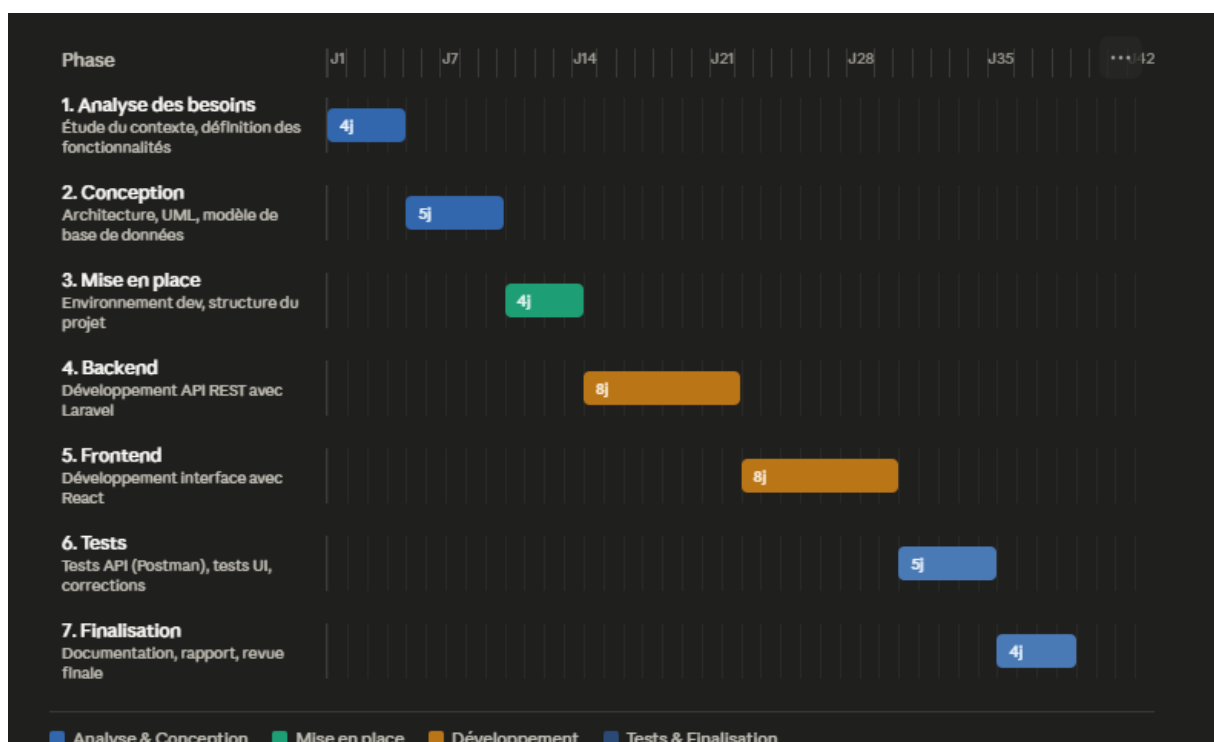


Figure 1:diagramme de Gantt de la plani 1

Chapitre 2 : Architecture et Conception du projet

1. Schéma global

L'architecture de TadlaBricol repose sur une approche en trois couches (architecture 3-tier), permettant de séparer clairement les responsabilités et d'améliorer la maintenabilité du système.

- Couche de présentation (Frontend) : application React/Vite, single page application.
- Couche logique (Backend / API REST) : Laravel 12 avec PHP 8.2.
- Couche de données : base de données relationnelle MySQL.

Cette organisation facilite les mises à jour et l'intégration de nouvelles fonctionnalités sans impacter les autres composants.

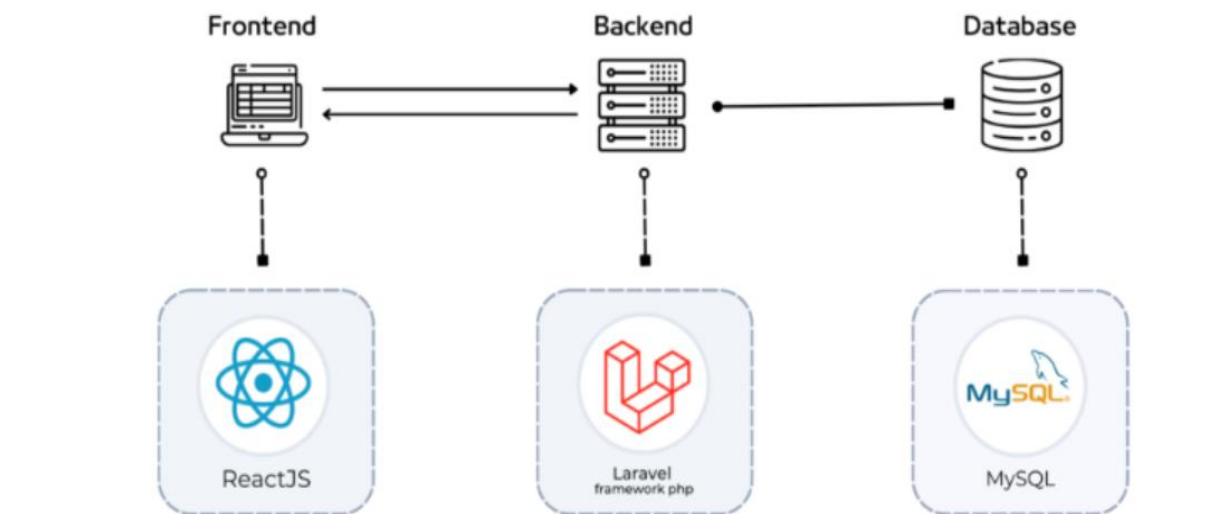


Figure 2: Schéma global de l'architecture du système

2. Plan du site et navigation (User Flow)

Le plan du site a été conçu pour garantir une navigation intuitive adaptée aux différents profils d'utilisateurs.

Pour un utilisateur non authentifié, l'accès se limite aux pages publiques : accueil, liste des artisans, profil artisan, connexion et inscription.

Une fois authentifié en tant que client, l'utilisateur peut :

- Parcourir et rechercher des artisans (filtres par profession et ville).
- Consulter les profils artisanaux avec services, photos et avis.
- Laisser un avis et une note sur un artisan.
- Ajouter/retirer un artisan de ses favoris.
- Envoyer et recevoir des messages via la messagerie intégrée.

L'artisan dispose d'un tableau de bord lui permettant de :

- Mettre à jour son profil (profession, description, ville, tarif horaire).
- Ajouter et supprimer des services.
- Uploader et gérer ses photos de réalisations.
- Définir sa disponibilité (disponible / occupé).

L'administrateur possède un accès complet incluant la gestion des utilisateurs et des artisans, la consultation des statistiques et la modération de la plateforme.

3. Diagrammes UML utiles

Dans le cadre de la conception du projet, les diagrammes UML ont été utilisés pour modéliser le système et représenter de manière claire les interactions entre ses composants.

Le diagramme de cas d'utilisation (Use Case) permet d'identifier les principaux acteurs du système — le client, l'artisan et l'administrateur — ainsi que les différentes fonctionnalités auxquelles ils ont accès. Ce diagramme met en évidence des actions telles que l'inscription, la connexion, la recherche d'artisans, la soumission d'avis, la gestion du profil artisan et l'administration des utilisateurs.

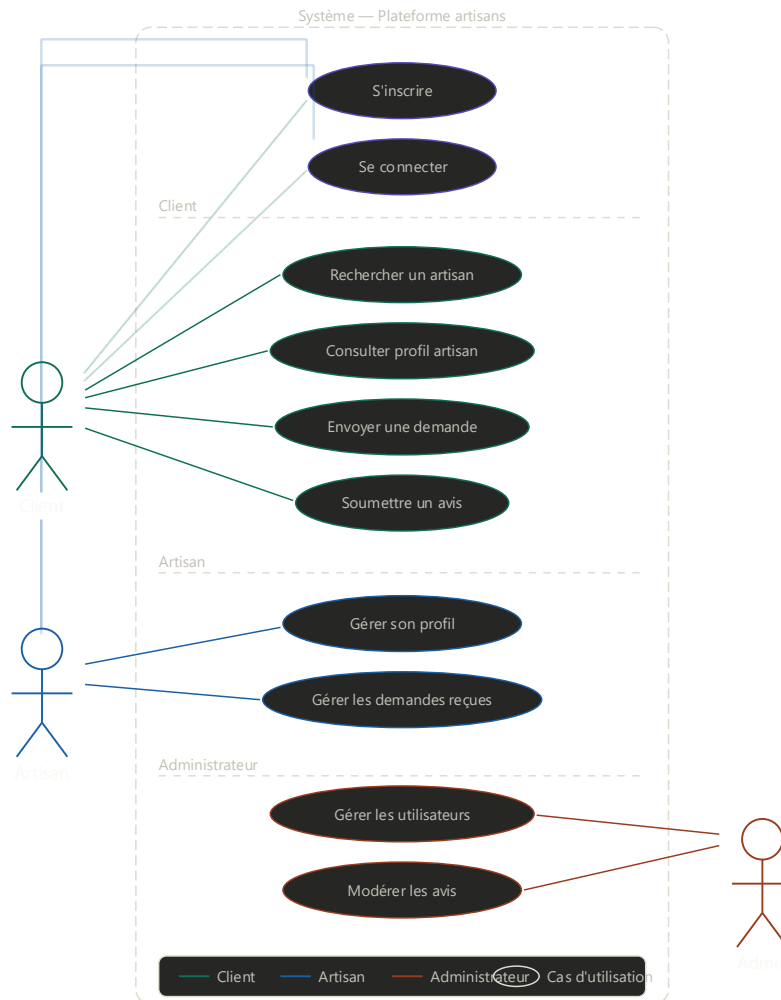


Figure 3:Diagramme de cas d'utilisation de la plateforme TadlaBricol

Le diagramme de séquence illustre le déroulement chronologique des interactions entre les composants. Par exemple, dans le scénario d'envoi d'un message, l'utilisateur saisit son message via le frontend, la requête est envoyée au backend via l'API REST, validée par le middleware Sanctum, puis enregistrée en base de données. Une réponse est ensuite renvoyée vers le frontend pour confirmer l'envoi.

Ces diagrammes sont présentés en détail dans l'Annexe 1 du présent rapport.

4. Conception de la base de données

La base de données est structurée autour des entités suivantes :

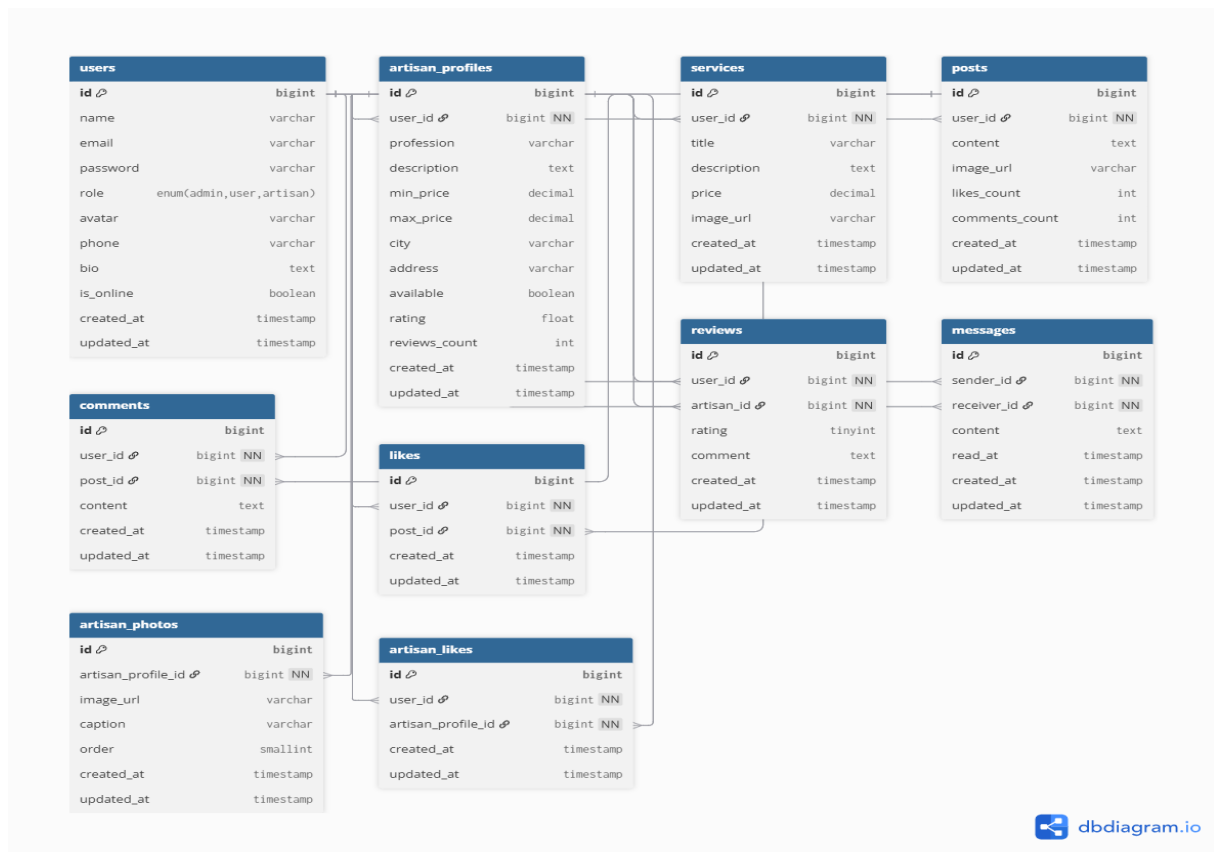


Figure 4: MCD du système TadlaBricol

Les principales entités du système sont :

- **users** : contient les informations des comptes utilisateurs (nom, email, mot de passe, rôle, etc.). Chaque utilisateur peut être un administrateur, un artisan ou un utilisateur simple.
- **artisan_profiles** : représente les profils des artisans, incluant leur profession, description, fourchette de prix, localisation et disponibilité.
- **services** : correspond aux services proposés par les artisans, avec un titre, une description, un prix et éventuellement une image.
- **posts** : permet aux utilisateurs de publier du contenu (texte et images) pour partager leurs réalisations ou expériences.
- **comments** : regroupe les commentaires publiés par les utilisateurs sur les posts.
- **likes** : enregistre les mentions "j'aime" des utilisateurs sur les publications.
- **reviews** : contient les avis et évaluations donnés par les utilisateurs sur les artisans.
- **messages** : gère les échanges de messages entre les utilisateurs (discussion privée).
- **artisan_photos** : stocke les photos associées aux profils des artisans (portfolio).
- **artisan_likes** : permet aux utilisateurs d'aimer ou de sauvegarder des profils d'artisans.

Chaque entité est associée à d'autres selon les relations suivantes :

- Un **artisan_profile** est lié à un utilisateur (artisan).
- Un **service** est créé par un utilisateur (artisan).
- Un **post** est publié par un utilisateur.
- Un **comment** est associé à un utilisateur et à un post.
- Un **like** est associé à un utilisateur et à un post.
- Un **review** est rédigé par un utilisateur pour un artisan.
- Un **message** relie deux utilisateurs (expéditeur et destinataire).
- Une **photo d'artisan** est liée à un profil d'artisan.
- Un **artisan_like** associe un utilisateur à un profil d'artisan.

5. Choix techniques et justification

Composant	Technologie	Justification
Frontend	React 19 + Vite	Composants réutilisables, rendu performant, HMR en développement
Routing	React Router	Navigation SPA sans rechargement de page
HTTP Client	Axios	Intercepteurs, gestion centralisée des tokens et erreurs
Styles	Tailwind CSS	Design responsive et cohérent avec classes utilitaires
Animations	Framer Motion	Transitions fluides et micro-interactions
i18n	i18next	Support multilingue FR/EN/AR avec gestion RTL/LTR
Icônes	Lucide React	Bibliothèque d'icônes légère et cohérente
Backend	Laravel 12 + PHP 8.2	Framework MVC robuste, fonctionnalités intégrées, Sanctum
Auth	Laravel Sanctum	Tokens révocables par session, adapté aux SPA
ORM	Eloquent	Requêtes préparées, protection contre les injections SQL
Base de données	MySQL	SGBD relationnel fiable pour données complexes

Chapitre 3 : Réalisation Technique du projet

1.Stack Frontend

Le frontend est développé avec React (via Vite), offrant une architecture orientée composants réutilisables, une compilation rapide et des performances optimisées en production.

L'application est organisée en pages et composants distincts :

Fichier / Dossier	Rôle
src/App.jsx	Point d'entrée, configuration du routing
src/pages/Home.jsx	Page d'accueil avec sections Hero, artisans vedettes, catégories
src/pages/Artisans.jsx	Listing des artisans avec recherche et filtres
src/pages/ArtisanProfile.jsx	Profil détaillé d'un artisan
src/pages/ArtisanDashboard.jsx	Tableau de bord artisan (gestion profil, services, photos)
src/pages/Inbox.jsx	Messagerie interne
src/pages/Admin/AdminDashboard.jsx	Tableau de bord administrateur
src/contexts/ThemeContext.jsx	Gestion du thème clair/sombre
src/i18n.js	Configuration i18next pour les 3 langues
src/translations/fr.js, en.js, ar.js	Fichiers de traduction

2. Stack Backend

Le backend est développé avec Laravel 12, exposant une API REST structurée selon le modèle MVC et sécurisée via Laravel Sanctum.

3.Architecture MVC :

- **Modèles** (Eloquent ORM) : User, ArtisanProfile, Service, Review, Message, ArtisanPhoto, ArtisanLike.
- **Contrôleurs** : gestion des requêtes, logique métier, retour des réponses JSON.

- **Routes** : définies dans routes/api.php, assurant la communication avec le frontend.

```
Route::middleware(['auth:sanctum'])->group(function () {

    Route::post('/logout', [AuthController::class, 'logout']);

    Route::get('/user', function (Request $request) {
        return $request->user();
    });

    Route::get('/users/{id}', function ($id) {
        $u = \App\Models\User::find($id);
        if (!$u) {
            return response()->json(['message' => 'Not found'], 404);
        }

        return response()->json([
            'id' => $u->id,
            'name' => $u->name,
            'phone' => $u->phone,
            'email' => $u->email
        ]);
    });

    // Messaging
    Route::get('/conversations', [MessageController::class, 'conversations']);
    Route::get('/conversations/{userId}/messages', [MessageController::class, 'messages']);
    Route::post('/messages', [MessageController::class, 'send']);

    // Artisan Routes
    Route::get('/artisan/profile', [ArtisanController::class, 'getProfile']);
    Route::post('/artisan/profile', [ArtisanController::class, 'updateProfile']);
    Route::post('/artisan/services', [ArtisanController::class, 'addService']);
    Route::delete('/artisan/services/{service}', [ArtisanController::class, 'deleteService']);

    Route::post('/artisan/photos/upload', [ArtisanController::class, 'uploadPhoto']);
    Route::post('/artisan/photos', [ArtisanController::class, 'addPhoto']);
    Route::delete('/artisan/photos/{photoId}', [ArtisanController::class, 'deletePhoto']);

    Route::post('/artisans/{id}/reviews', [ArtisanController::class, 'storeReview']);
    Route::get('/me/artisan-likes', [ArtisanController::class, 'likedArtisans']);
    Route::post('/artisans/{id}/like', [ArtisanController::class, 'like']);
    Route::delete('/artisans/{id}/like', [ArtisanController::class, 'unlike']);

    // Admin Routes
    Route::middleware('admin')->prefix('admin')->group(function () {
        Route::get('/stats', [AdminController::class, 'stats']);
        Route::get('/users', [AdminController::class, 'users']);
        Route::get('/artisans', [AdminController::class, 'artisans']);
        Route::put('/users/{user}', [AdminController::class, 'updateUser']);
        Route::delete('/users/{user}', [AdminController::class, 'deleteUser']);
    });
});
```

Les routes sensibles de l'application sont protégées par le middleware auth:sanctum. Cela signifie que l'utilisateur doit être authentifié avec un token valide pour accéder aux fonctionnalités privées comme la messagerie, le profil artisan et le tableau de bord

administrateur.

```
class EnsureUserIsAdmin
{
    public function handle(Request $request, Closure $next): Response
    {
        if (!$request->user() || $request->user()->role !== 'admin') {
            return response()->json(['message' => 'Unauthorized. Admin only.'], 403);
        }

        return $next($request);
    }
}
```

En plus de Sanctum, les routes administrateur sont protégées par un middleware personnalisé admin. Ce middleware vérifie que l'utilisateur connecté possède le rôle admin. Si ce n'est pas le cas, l'accès est refusé avec une réponse 403.

4.Authentication — Laravel Sanctum

Toutes les routes protégées requièrent un token Bearer valide (middleware : auth:sanctum). Le token est émis à la connexion et invalidé à la déconnexion, permettant une révocation instantanée contrairement au JWT.

Aspect	Laravel Sanctum	JWT
Stockage	Base de données	Autonome (signé)
Révocation	Instantanée	Impossible sans blacklist
Complexité	Simple	Plus complexe
Adapté aux SPA	Oui	Oui

5.API REST

Route	Méthode	Description	Auth
/auth/register	POST	Inscription d'un nouvel utilisateur	Non
/auth/login	POST	Connexion et récupération du token	Non
/auth/logout	POST	Déconnexion (suppression du token)	Oui
/auth/forgot-password	POST	Demande de réinitialisation du mot de passe	Non
/auth/reset-password	POST	Réinitialisation du mot de passe	Non
/artisans	GET	Liste publique des artisans avec recherche/filtres	Non
/artisans/{id}	GET	Profil public d'un artisan	Non
/artisan/profile	PUT	Mise à jour du profil artisan	Oui
/artisan/services	POST/ DELETE	Ajout/suppression de services	Oui
/artisan/photos	POST/ DELETE	Gestion des photos de réalisations	Oui
/artisan/availability	PUT	Définir disponibilité (available/busy)	Oui
/artisans/{id}/reviews	POST	Soumettre un avis sur un artisan	Oui
/artisans/{id}/like	POST	Aimer/ne plus aimer un artisan	Oui
/likes	GET	Liste des artisans favoris	Oui
/conversations	GET	Liste des conversations	Oui
/messages/{userId}	GET/ POST	Messages avec un utilisateur	Oui
/admin/stats	GET	Statistiques du tableau de bord admin	Admin
/admin/users	GET/ PUT/ DELETE	Gestion des utilisateurs	Admin

6. Gestion des rôles et sécurité

Le système implémente un contrôle d'accès basé sur les rôles (RBAC), où chaque utilisateur n'accède qu'aux ressources correspondant à son rôle.

Mécanisme	Implémentation
Authentification	Laravel Sanctum — tokens par session, révocables
Autorisation	Middleware <code>EnsureUserIsAdmin</code> + vérification du rôle dans les contrôleurs
Protection des routes	Middleware <code>auth:sanctum</code> sur toutes les routes privées
Validation des entrées	Validation Laravel — rejet des données invalides avant traitement
Protection SQL	Eloquent ORM — requêtes préparées, pas de SQL brut

Chapitre 4 : Implémentation par Fonctionnalité

1.Parcours Client

Le parcours client a été conçu pour garantir une expérience simple et intuitive. Après authentification, l'utilisateur accède à la liste des artisans qu'il peut parcourir, filtrer et rechercher.

Sur la page de profil d'un artisan, le client peut :

- Consulter les informations de l'artisan (profession, ville, tarif, description).
- Voir les services proposés et les photos de réalisations.
- Lire les avis et la note moyenne des clients précédents.
- Laisser un avis (une seule fois par artisan, avec note de 1 à 5).
- Envoyer un message directement à l'artisan.
- Ajouter l'artisan à ses favoris.

2.Parcours Artisan

L'artisan dispose d'un tableau de bord complet permettant de gérer intégralement sa présence sur la plateforme. Depuis cet espace, il peut mettre à jour ses informations professionnelles, gérer ses services et ses photos de réalisations, et définir sa disponibilité.

Le système de notes et d'avis mis à jour automatiquement après chaque avis client renforce la crédibilité de l'artisan et sa visibilité sur la plateforme.

3.Parcours Administrateur

L'administrateur dispose d'un tableau de bord centralisé offrant une vue d'ensemble sur la plateforme. Ses responsabilités incluent :

- Gestion complète des comptes utilisateurs (consultation, modification du rôle, suppression).
- Gestion des profils artisans.
- Consultation des statistiques globales de la plateforme.
- Supervision de l'activité générale.

Par mesure de sécurité, l'administrateur ne peut pas supprimer son propre compte.

4.Messagerie

Le système de messagerie permet aux utilisateurs d'échanger directement avec les artisans. Les fonctionnalités incluent la consultation des conversations, l'envoi de messages et le marquage automatique des messages comme lus lors de leur ouverture.

5.Internationalisation et accessibilité

La plateforme intègre nativement un support multilingue via i18next, couvrant le français, l'anglais et l'arabe. La gestion bidirectionnelle (RTL/LTR) adapte automatiquement l'interface pour les utilisateurs arabophones.

- Architecture trilingue : fichiers de traduction séparés (fr.js, en.js, ar.js).
- Adaptation automatique de la direction de lecture selon la langue choisie.
- Interface responsive compatible mobile, tablette et desktop.
- Thème clair/sombre géré via ThemeContext.

6.Gestion des médias

L'artisan peut uploader des photos de ses réalisations pour enrichir son profil. Les fichiers sont validés (type et taille), stockés de manière sécurisée et accessibles publiquement depuis le profil artisan.

7.Gestion des erreurs et notifications

Le système de toast (ToastContext) informe l'utilisateur de toutes les actions : succès, erreurs de validation, et messages d'état. Les erreurs API sont gérées de manière centralisée via les intercepteurs Axios, avec redirection automatique en cas de token expiré.

Chapitre 5 : Technologies et Outils Utilisés

1. Technologies Frontend

HTML5 Pour structurer les pages web avec des éléments sémantiques.	HTML 
	CSS3 Pour styliser les pages web et créer des mises en page réactives.
Tailwind CSS Framework CSS pour faciliter la création de designs réactifs.	
	JavaScript Pour rendre les pages web interactives.
React Bibliothèque JavaScript moderne utilisée pour la création d'interfaces utilisateur interactives et dynamiques. Elle repose sur une architecture basée sur des composants réutilisables, permettant de construire des applications web performantes et évolutives.	

2. Technologies Backend

Laravel Framework PHP moderne et élégant, conçu pour faciliter le développement d'applications web robustes. Il propose une syntaxe expressive et des outils intégrés puissants pour gérer efficacement les bases de données, le routage, l'authentification et les migrations de données.	
	MySQL Système de gestion de bases de données relationnelles pour stocker et gérer les données du système de gestion des réclamations

3. Outils de développement

3.1 Environnement de Développement

Visual Studio Code Éditeur de code extensible pour le développement et le débogage du code.	
	Xampp Outil de développement incluant Apache, MySQL et PHP pour tester l'application localement

3.2.Outils de Gestion de Projet

GanttProject Logiciel open-source de gestion de projet utilisé pour la planification et l'ordonnancement des tâches.	
---	---

3.4.Outils de Test

Postman Outil de développement utilisé pour tester et valider les endpoints de l'API en simulant des requêtes HTTP.	
--	---

3.5Outils de Gestion de Version

GitHub GitHub est une plateforme de développement collaboratif qui permet de gérer le code source et de suivre les modifications apportées au projet.		
	Git Système de gestion de versions décentralisé permettant de suivre l'évolution du code source.	

Chapitre 6 : Présentation des interfaces

1.Introduction

Ce chapitre décrit les principales interfaces de la plateforme, en mettant l'accent sur leur organisation, leur lisibilité et leur facilité d'utilisation. Chaque page a un rôle précis dans le parcours utilisateur et assure une navigation fluide.

2.Page d'accueil

Point d'entrée de la plateforme, la page d'accueil présente le projet et oriente l'utilisateur vers les principales actions. Elle se compose d'une section Hero avec appel à l'action, d'une section présentant les catégories de services disponibles, d'une section mettant en avant les artisans vedettes et d'une section de réassurance (Pourquoi nous faire confiance).

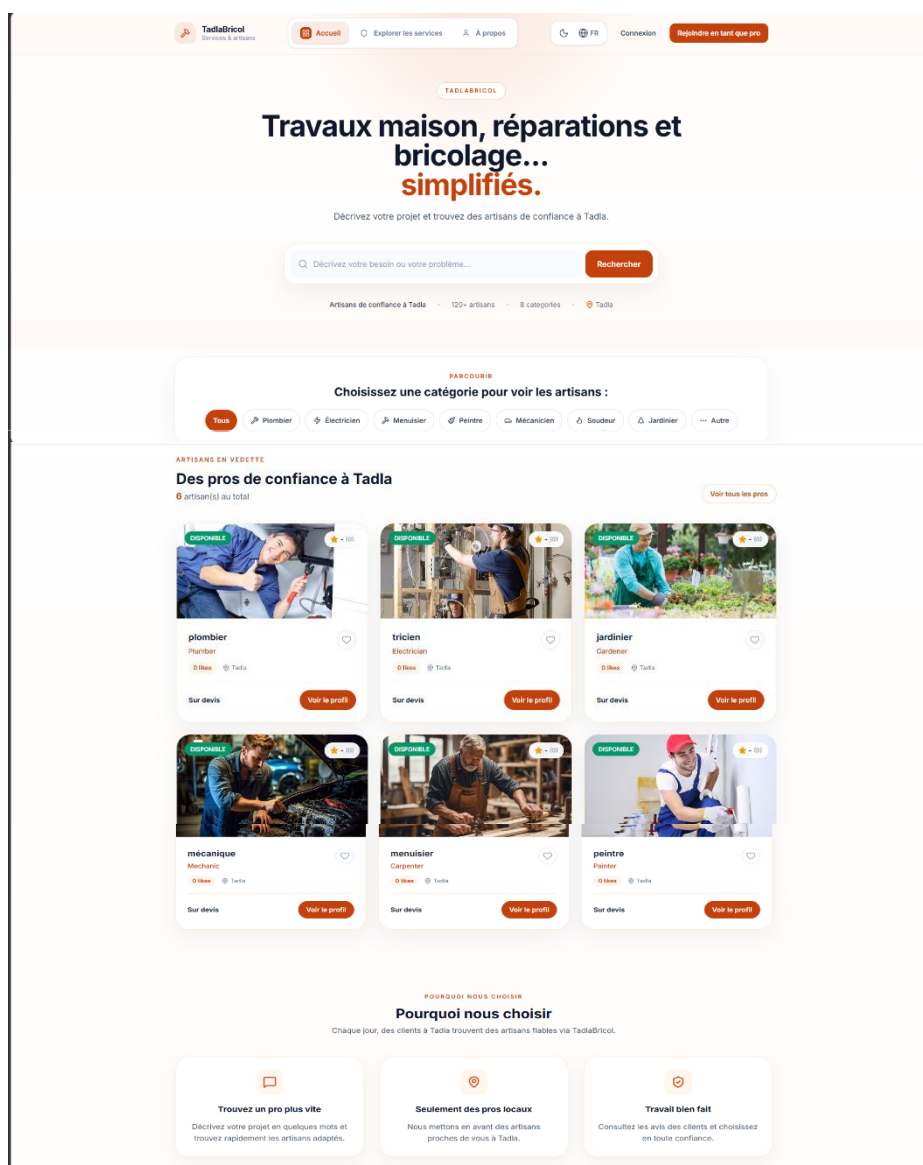
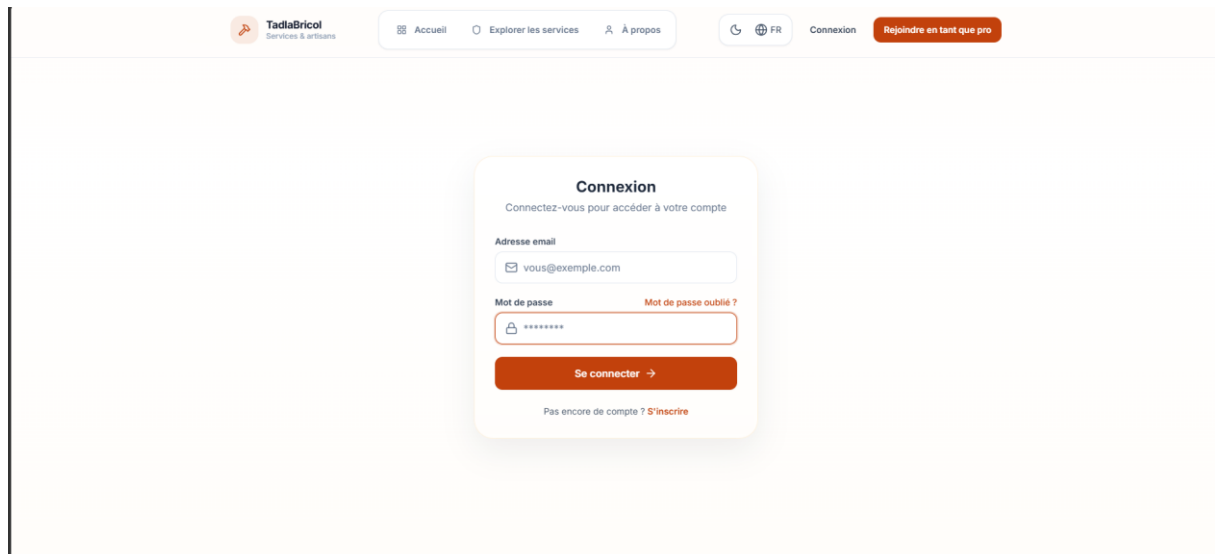


Figure 5:Page d'accueil

3. Page de connexion

La page de connexion permet aux utilisateurs déjà inscrits d'accéder à leur compte en saisissant leur adresse e-mail et leur mot de passe. Elle représente le point d'entrée vers les espaces privés du site, comme le tableau de bord artisan, la messagerie ou l'espace administrateur.



The screenshot shows the login page for 'TadiaBricol Services à artisans'. The header includes navigation links: 'Accueil', 'Explorer les services', 'À propos', a language selector set to 'FR', and a 'Connexion' button. A red button labeled 'Rejoindre en tant que pro' is also present. The main content area features a central login form titled 'Connexion' with the instruction 'Connectez-vous pour accéder à votre compte'. The form has two input fields: 'Adresse email' (containing 'vous@exemple.com') and 'Mot de passe' (masked with '*****'). A link for 'Mot de passe oublié ?' is next to the password field. Below the inputs is a red 'Se connecter →' button. At the bottom of the form, it says 'Pas encore de compte ? S'inscrire'.

Figure 6: Page de connexion

4 Page d'inscription

Cette page permet à un nouvel utilisateur de créer un compte sur la plateforme. Lors de l'inscription, il peut choisir son rôle, notamment client ou artisan. Cette interface est importante car elle permet d'alimenter la plateforme en nouveaux utilisateurs et professionnels.

The image displays two versions of the 'Créer un compte' (Create account) page on the TadiaBricol website. The page layout includes a header with the logo, navigation links (Accueil, Explorer les services, À propos), language selection (FR), and a 'Connexion' button. A 'Rejoindre en tant que pro' button is also present.

Top Screenshot (Je suis client selected):

- Créer un compte**
Rejoignez la plateforme artisanale de référence à Tadia
- Buttons: **Je suis client** (selected), **Je suis artisan**
- Form fields:
 - Nom complet: Jean Dupont
 - Adresse email: peintre@gmail.com
 - Numéro de téléphone: +212 6...
 - Mot de passe: [masked]
 - Confirmer: [masked]
- Buttons: **Créer un compte →**
- Text: Vous avez déjà un compte ? [Se connecter](#)

Bottom Screenshot (Je suis artisan selected):

- Créer un compte**
Rejoignez la plateforme artisanale de référence à Tadia
- Buttons: **Je suis client**, **Je suis artisan** (selected)
- Form fields:
 - Nom complet: Jean Dupont
 - Adresse email: peintre@gmail.com
 - Numéro de téléphone: +212 6...
 - Profession: Sélectionnez votre profession
 - Mot de passe: [masked]
 - Confirmer: [masked]
- Buttons: **Créer un compte →**

Figure 7: Page d'inscription

5. Section des artisans disponibles

Cette interface présente une sélection d'artisans disponibles sur la plateforme TadlaBricol. Chaque carte affiche les informations essentielles de l'artisan, telles que son nom, sa profession, sa localisation, sa disponibilité ainsi que l'accès à son profil détaillé. Cette section permet à l'utilisateur d'avoir un aperçu rapide des professionnels présents sur la plateforme et facilite l'accès aux profils des artisans.

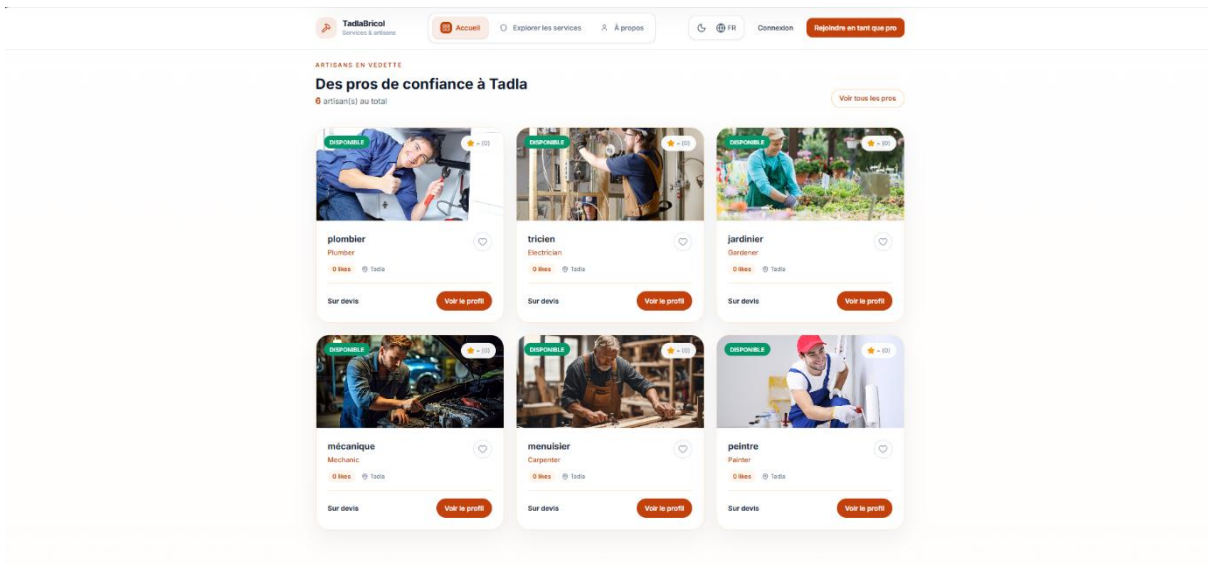


Figure 8: Section des artisans disponibles

6. Page de profil d'un artisan

La page de profil présente en détail les informations relatives à un artisan. On y trouve sa description, ses services, ses photos, ses avis clients ainsi que les moyens de le contacter. Cette page aide le client à prendre une décision avant de choisir un professionnel.

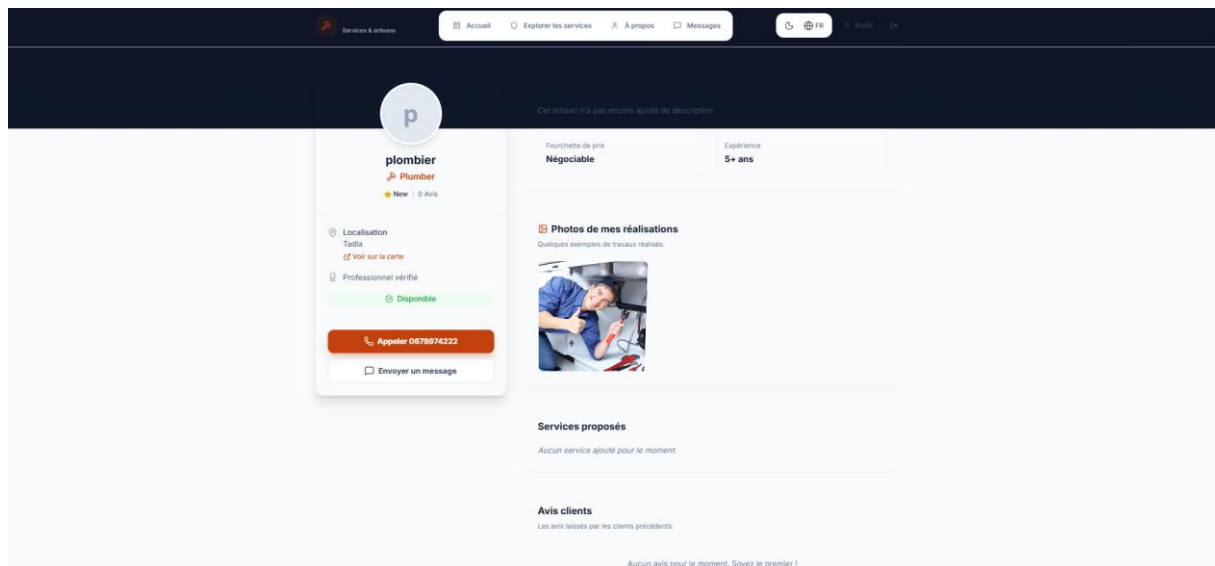


Figure 9: Page de profil d'un artisan

7. Tableau de bord artisan

Cette page est réservée aux artisans connectés. Elle leur permet de gérer leur espace personnel, mettre à jour leurs informations professionnelles, ajouter des services et publier des photos de leurs réalisations. Cette interface constitue l'outil principal de gestion de l'activité de l'artisan sur la plateforme

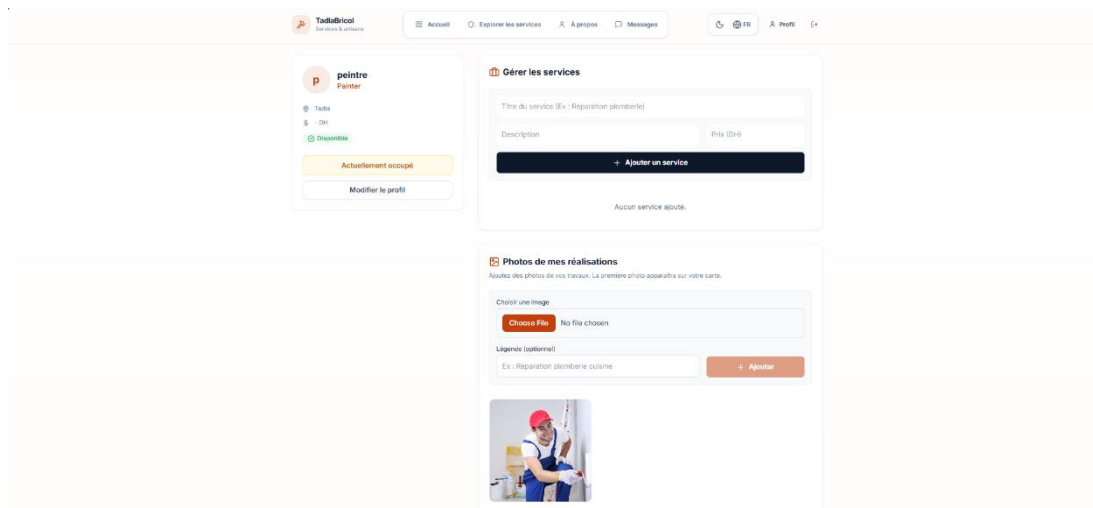


Figure 10: Tableau de bord artisan

8. Page de messagerie

La messagerie permet aux utilisateurs d'échanger directement avec les artisans via des conversations privées. Cette fonctionnalité facilite la communication, la demande d'informations et la coordination entre les deux parties.

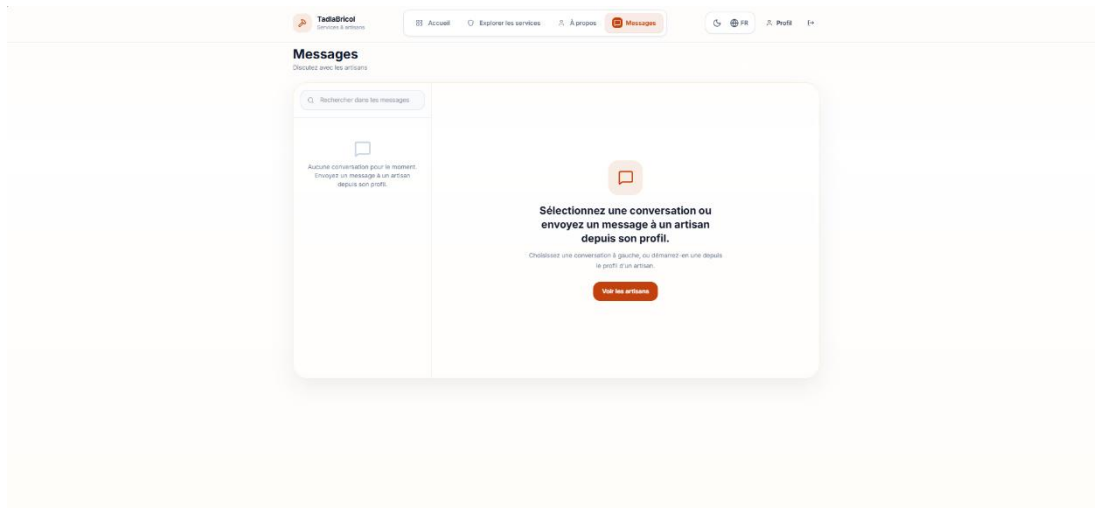


Figure 11: Page de messagerie

9. Tableau de bord administrateur - Vue d'ensemble

Cette interface permet à l'administrateur d'obtenir une vue globale sur les principales statistiques de la plateforme, notamment le nombre d'utilisateurs, d'artisans, de profils et de services.

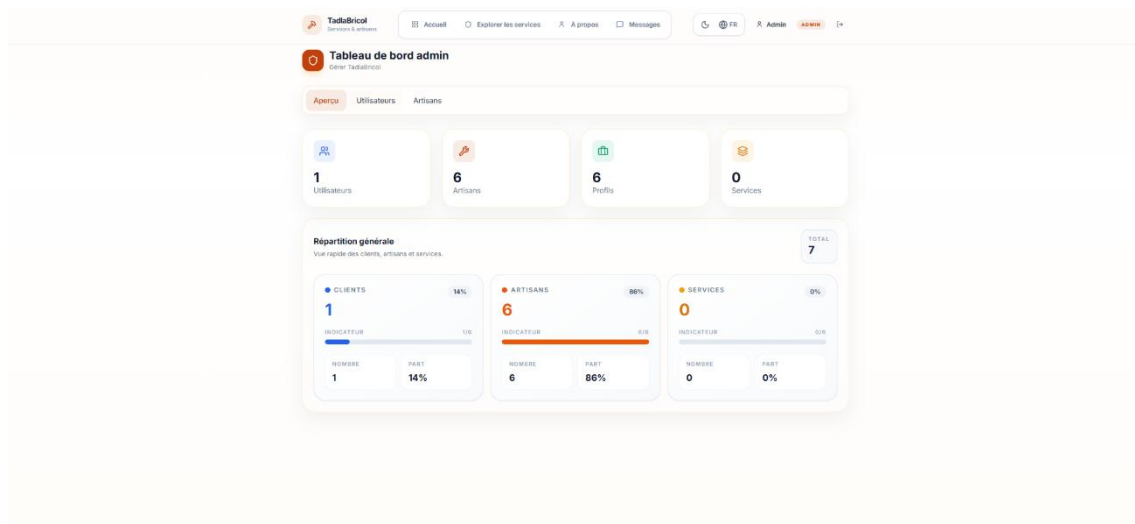


Figure 12: Tableau de bord administrateur - Vue d'ensemble

10. Tableau de bord administrateur - Gestion des utilisateurs

Cette interface permet à l'administrateur de consulter les utilisateurs inscrits sur la plateforme et de suivre les informations principales liées à leurs comptes.

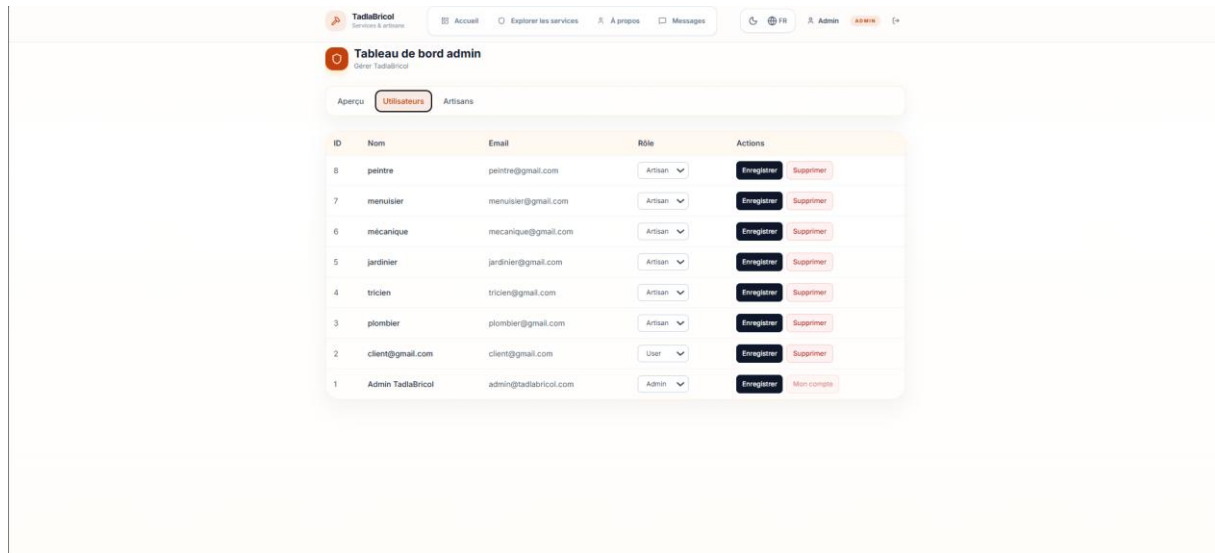
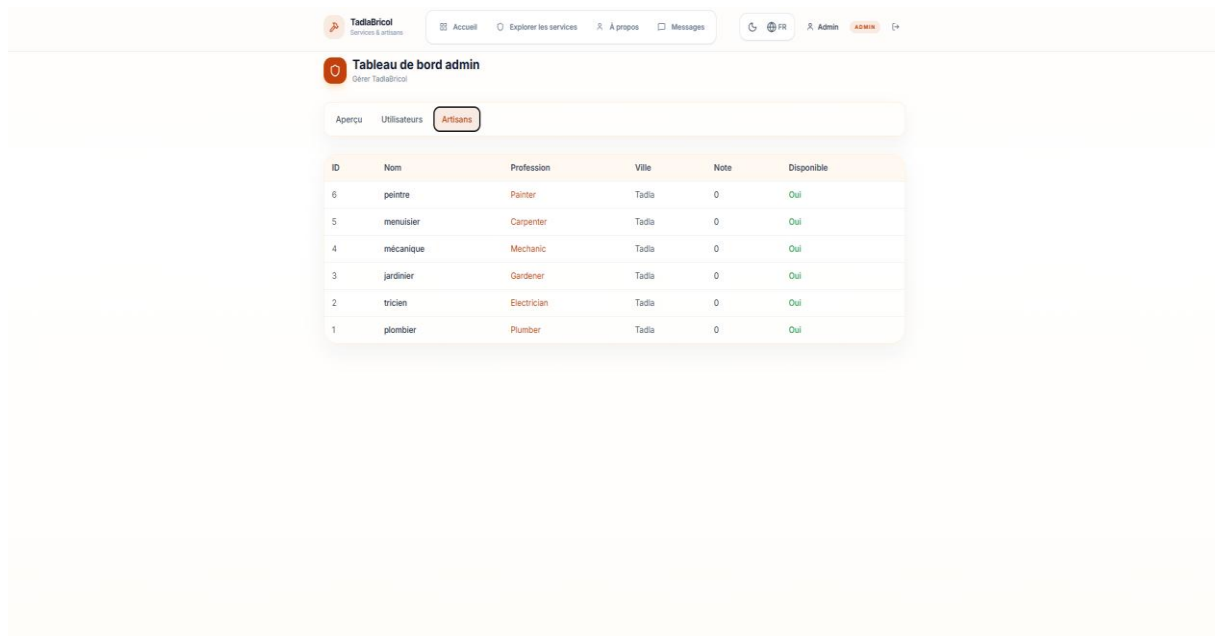


Figure 13: Tableau de bord administrateur - Gestion des utilisateurs

11. Tableau de bord administrateur - Gestion des artisans

Cette interface permet à l'administrateur de visualiser les artisans enregistrés sur la plateforme, leurs informations professionnelles, leur disponibilité et certaines données utiles à la supervision du système



The screenshot displays the 'Tableau de bord admin' (Admin Dashboard) for 'TadiaBricol Services & artisans'. The dashboard includes a navigation bar with links to 'Accueil', 'Explorer les services', 'À propos', and 'Messages'. Below the navigation bar, there are tabs for 'Aperçu', 'Utilisateurs', and 'Artisans', with 'Artisans' being the active tab. The main content area shows a table of registered artisans with the following data:

ID	Nom	Profession	Ville	Note	Disponible
6	peintre	Painter	Tadia	0	Oui
5	menuisier	Carpenter	Tadia	0	Oui
4	mécanique	Mechanic	Tadia	0	Oui
3	jardinier	Gardener	Tadia	0	Oui
2	électricien	Electrician	Tadia	0	Oui
1	plombier	Plumber	Tadia	0	Oui

Figure 14: Tableau de bord administrateur - Gestion des artisans

Chapitre 7 : Tests et Validation

1.Stratégie de test

La phase de test est essentielle pour s'assurer du bon fonctionnement de l'ensemble des fonctionnalités de la plateforme. La stratégie adoptée repose sur des scénarios de tests manuels reproduisant les actions réelles des utilisateurs selon leurs rôles (client, artisan, administrateur).

Les principaux objectifs étaient :

- Vérifier la création de compte et l'authentification sécurisée.
- Tester la recherche et le filtrage des artisans.
- Contrôler le système d'avis avec ses contraintes (une fois par client, pas d'auto-avis).
- Vérifier le bon fonctionnement de la messagerie.
- Contrôler la gestion des rôles et les redirections.
- Vérifier les fonctionnalités du tableau de bord artisan.
- Tester les fonctionnalités administratives.

2.Outils de test

2.1Tests de l'API avec Postman

- Vérification des endpoints : authentification, artisans, avis, messagerie, admin.
- Contrôle de la validité des réponses et des formats JSON.
- Test des contrôles d'accès (routes protégées, vérification des rôles).
- Détection des erreurs de validation et correction pour améliorer la fiabilité.

2.2. Tests de l'interface utilisateur (UI)

- Vérification de l'affichage correct de toutes les pages et de la fluidité de navigation.
- Contrôle des formulaires, boutons, messages d'erreur et notifications toast.
- Test du support multilingue (FR/EN/AR) et du basculement RTL/LTR.
- Vérification de la responsivité sur différentes tailles d'écran.

3. Résultats et corrections

Aspect testé	Résultat	Correction apportée
Authentification	OK	Gestion des cas d'erreur améliorée
Recherche artisans	OK	Optimisation des requêtes
Système d'avis	OK	Validation des contraintes côté backend
Messagerie	OK	Marquage lu à l'ouverture
Contrôle d'accès admin	OK	Middleware EnsureUserIsAdmin renforcé
Support multilingue	OK	Correction de textes non traduits
Responsive mobile	Partiel	Améliorations en cours

Chapitre 8 : Déploiement et Exploitation

1. Environnement de développement local

Composant	Outil / Version
Système d'exploitation	Windows 11
Éditeur de code	Visual Studio Code
Serveur local	XAMPP (Apache + MySQL)
Runtime backend	PHP 8.2
Framework backend	Laravel 12
Runtime frontend	Node.js 20.x
Framework frontend	React 19 + Vite 6
Base de données	MySQL 8.0
Versioning	Git + GitHub
Tests API	Postman

2. Plan de déploiement

Même si le projet est actuellement en phase de développement local, un plan de déploiement est envisagé pour une mise en production future :

- Frontend : build de production via 'npm run build', déploiement sur une plateforme d'hébergement statique ou CDN.
- Backend : déploiement sur un serveur Apache/Nginx avec PHP 8.2, configuration des variables d'environnement (.env) et des permissions de fichiers.
- Base de données : MySQL sur serveur dédié avec sauvegardes automatiques régulières.
- Configuration HTTPS en production pour sécuriser les communications.

3.Sécurité et sauvegardes envisagées

- Authentification et gestion des rôles via Laravel Sanctum.
- Protection HTTPS en production.
- Variables d'environnement non committées (APP_KEY, DB_PASSWORD dans .env).
- URL de l'API configurée via VITE_API_URL pour faciliter le passage dev/prod.
- Sauvegardes régulières et automatiques de la base de données.
- Maintenance continue pour corrections de bugs et mises à jour de dépendances.

Chapitre 9 : Résultats, Limites et Perspectives

1. Résultats obtenus

Le projet TadlaBricol a permis de développer une plateforme web complète de mise en relation artisanale. Les principaux résultats obtenus montrent que les objectifs ont été atteints :

- Pour les clients : recherche efficace d'artisans avec filtres, consultation de profils complets, système d'avis fiable, messagerie intégrée, gestion des favoris.
- Pour les artisans : tableau de bord complet pour gérer leur présence numérique, visibilité accrue, gestion des services et photos.
- Pour l'administration : supervision complète de la plateforme, gestion des utilisateurs.
- Support multilingue (FR/EN/AR) avec gestion RTL/LTR opérationnelle.

2. Limites actuelles

Malgré les résultats obtenus, certaines limites subsistent et constituent des axes d'amélioration:

Limite	Impact	Priorité
URL API en dur dans certains fichiers frontend	Passage dev/prod difficile	Haute
Absence de notifications en temps réel (WebSocket)	Expérience messagerie limitée	Haute
Token de reset exposé dans la réponse API (mode dev)	Risque sécurité en production	Haute
Optimisation mobile partielle	UX mobile perfectible	Moyenne
Absence de pagination admin	Performance dégradée avec beaucoup d'utilisateurs	Moyenne
Doublons possibles dans migrations photos	Risque d'erreurs	Faible
Couverture de tests backend insuffisante	Risque régressions	Moyenne

3.Perspectives d'évolution

Plusieurs améliorations peuvent être envisagées pour les versions futures :

- Configurer l'URL de l'API via la variable d'environnement VITE_API_URL.
- Intégration de notifications en temps réel (WebSocket / Laravel Echo).
- Sécuriser le reset de mot de passe en envoyant le token par email.
- Amélioration de la pagination et des filtres dans le tableau de bord administrateur.

- Vérification et correction des migrations artisan_photos en double.
- Ajout de tests automatisés backend (PHPUnit / Pest).
- Amélioration de la responsivité mobile (design adaptatif).
- Développement d'une application mobile compagnon (React Native).
- Intégration d'un système de géolocalisation pour rechercher des artisans à proximité.

Conclusion

Au terme de ce projet de stage, la plateforme TadlaBricol s'impose comme une réponse concrète et structurée au besoin de valorisation numérique de l'artisanat local dans la région de Tadla. En centralisant l'offre artisanale au sein d'un système unique, l'application améliore significativement la visibilité des artisans, facilite la mise en relation avec les clients et renforce la confiance grâce au système d'avis.

Sur le plan technique, ce projet a mobilisé un ensemble de technologies modernes — React, Laravel, MySQL, il8next — organisées selon une architecture en trois couches garantissant la séparation des responsabilités, la sécurité et l'évolutivité du système. Les phases de conception, de développement et de test ont permis de livrer une solution fonctionnelle couvrant l'ensemble des besoins identifiés.

Sur le plan personnel, cette expérience a été particulièrement enrichissante. Elle a permis de confronter les connaissances acquises en formation aux exigences réelles d'un environnement professionnel, et de développer une approche méthodique et rigoureuse du développement web full stack.

Les limites identifiées et les perspectives d'évolution tracées constituent une feuille de route claire pour les améliorations futures, témoignant du potentiel évolutif de la plateforme.

Bilan

Compétences développées

Ce stage a permis de consolider et d'approfondir un ensemble de compétences techniques et humaines :

Sur le plan technique :

- Maîtrise de React 19 avec Vite pour le développement d'interfaces modernes.
- Développement d'API REST robustes avec Laravel 12 et PHP 8.2.
- Mise en place d'une authentification sécurisée via Laravel Sanctum.
- Implémentation d'un système multilingue complet (i18next, RTL/LTR).
- Utilisation de Git pour la gestion de versions et la collaboration.
- Test et validation d'API avec Postman.

Sur le plan humain et professionnel :

- Développement de l'autonomie et de la rigueur dans la résolution de problèmes techniques.
- Capacité d'adaptation à un environnement professionnel réel.
- Amélioration de la communication et de la documentation technique.

Problèmes rencontrés et solutions apportées

Plusieurs défis techniques ont été rencontrés au cours du développement. La mise en place de l'authentification dans une architecture SPA séparée a nécessité une compréhension approfondie de Laravel Sanctum et de la gestion des tokens CSRF.

La mise en place du support bilingue français/arabe, notamment la gestion des interfaces en mode RTL/LTR avec Tailwind CSS, a constitué un défi au niveau de la conception. Ce problème a été résolu en adaptant les layouts et en utilisant les fonctionnalités de direction d'écriture du framework.

Ces difficultés ont été surmontées grâce à la consultation de la documentation officielle, aux recherches personnelles et aux échanges avec les encadrants.

Bibliographie

Documentation officielle :

- Documentation officielle de Laravel : <https://laravel.com>
- Documentation officielle de React : <https://react.dev>
- Documentation officielle d'Axios : <https://axios-http.com>
- Documentation officielle de Tailwind CSS : <https://tailwindcss.com>
- Documentation officielle de i18next : <https://www.i18next.com>
- Documentation officielle de Framer Motion : <https://www.framer.com/motion>
- Documentation officielle de MySQL : <https://dev.mysql.com>
- Documentation officielle de Postman : <https://www.postman.com>
- Documentation officielle de GitHub : <https://docs.github.com>

Annexes

Diagrammes UML

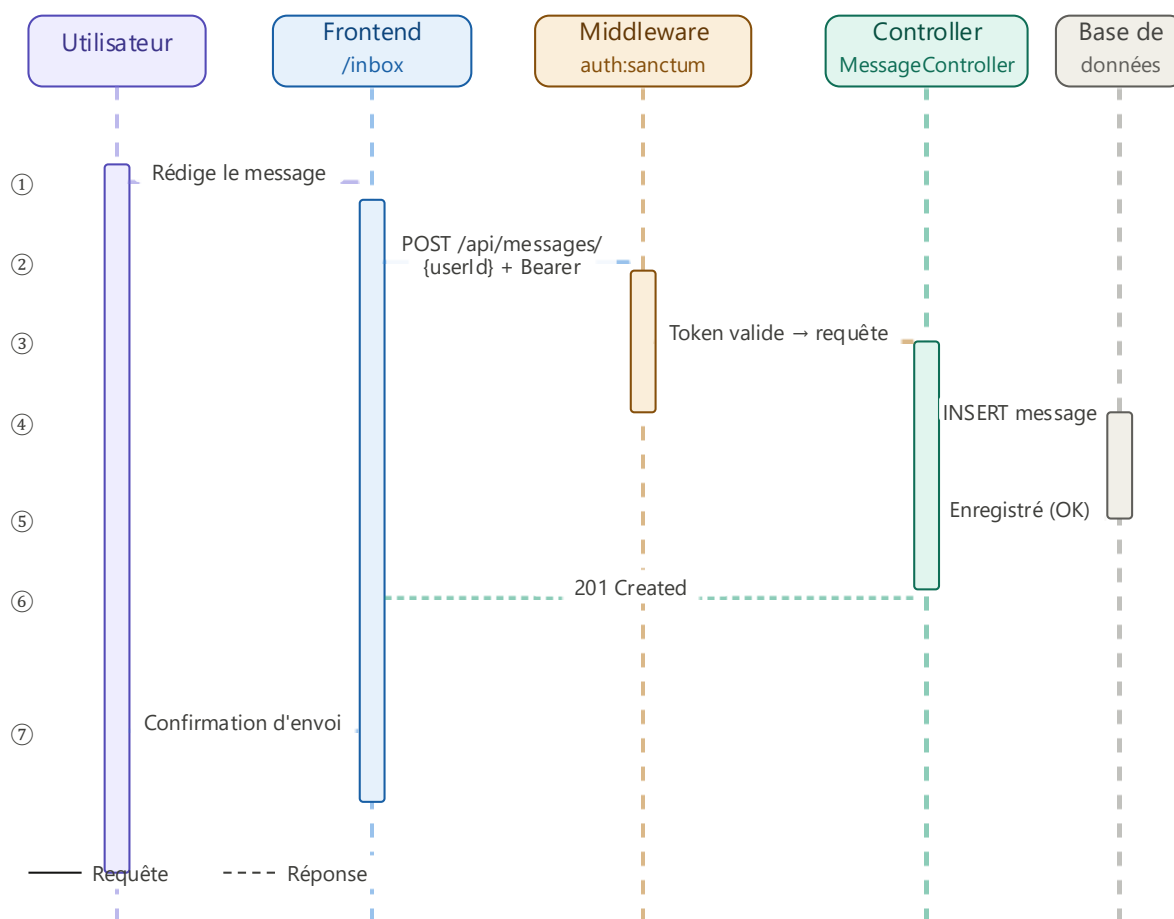


Figure 15:Diagramme de séquence — Envoi d'un message

Diagramme de classes

Le diagramme de classes présente les principales entités du système et leurs relations :

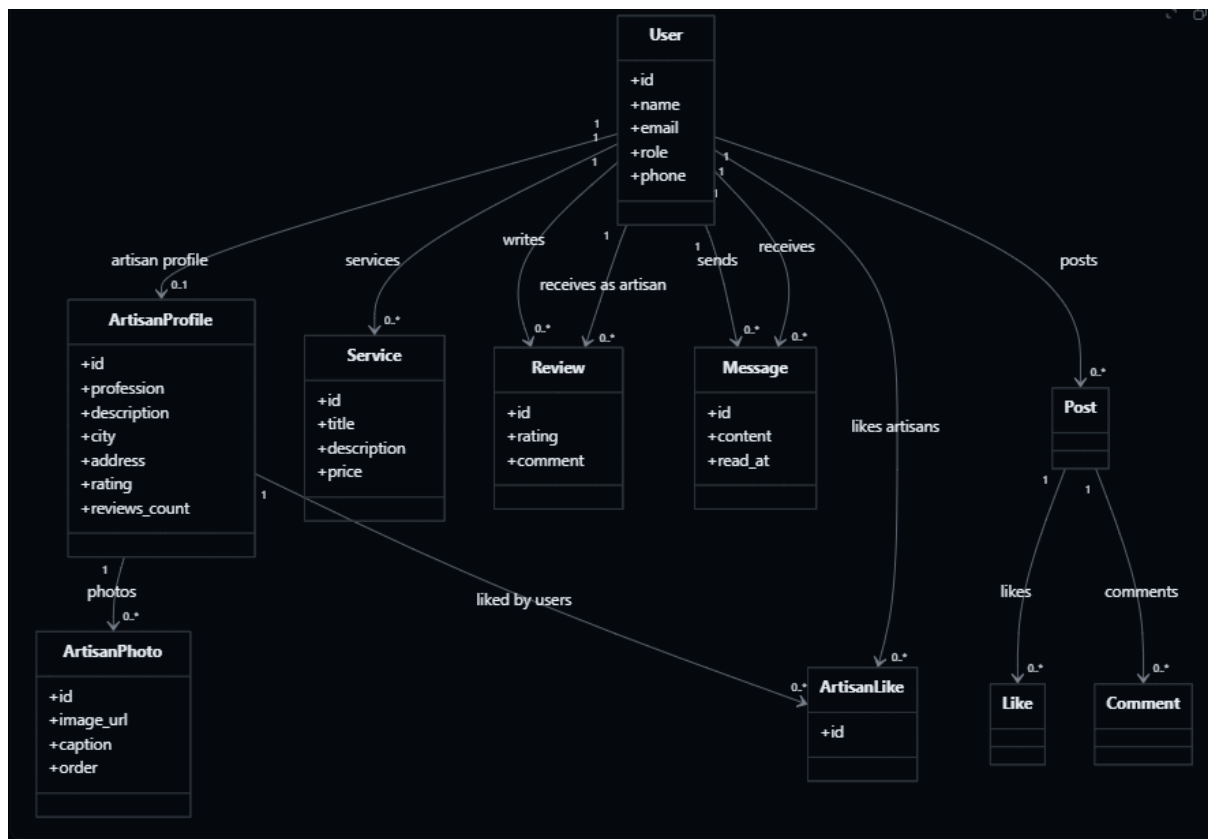


Figure 16: Diagramme de classes

Configuration de l'environnement

Configuration backend (.env) :

```
backend > .env
1 APP_NAME=Laravel
2 APP_ENV=local
3 APP_KEY=base64:Cy88F2RL8XZCwKt5SQDwdIw8J7YK2X06710P10=
4 APP_DEBUG=true
5 APP_URL=http://localhost:8000
6
7 APP_LOCALE=en
8 APP_FALLBACK_LOCALE=en
9 APP_FAKER_LOCALE=en_US
10
11 APP_MAINTENANCE_DRIVER=file
12 # APP_MAINTENANCE_STORE=database
13
14 # PHP_CLI_SERVER_WORKERS=4
15
16 BCRYPT_ROUNDS=12
17
18 LOG_CHANNEL=stack
19 LOG_STACK=single
20 LOG_DEPRECATIONS_CHANNEL=null
21 LOG_LEVEL=debug
22
23 DB_CONNECTION=mysql
24 DB_HOST=127.0.0.1
25 DB_PORT=3306
26 DB_DATABASE=tadabricole
27 DB_USERNAME=root
28 DB_PASSWORD=
29
30 SESSION_DRIVER=database
31 SESSION_LIFETIME=120
32 SESSION_ENCRYPT=false
33 SESSION_PATH=/
34 SESSION_DOMAIN=null
35
36 BROADCAST_CONNECTION=log
37 FILESYSTEM_DISK=local
38 QUEUE_CONNECTION=database
39
40 CACHE_STORE=database
41 # CACHE_PREFIX=
42
43 MEMCACHED_HOST=127.0.0.1
44
45 REDIS_CLIENT=phpredis
46 REDIS_HOST=127.0.0.1
47 REDIS_PASSWORD=null
48 REDIS_PORT=6379
49
50 MAIL_MAILER=log
51 MAIL_SCHEME=null
52 MAIL_HOST=127.0.0.1
53 MAIL_PORT=2525
```

Configuration frontend (.env) :

```
1 {
2   "name": "frontend",
3   "private": true,
4   "version": "0.0.0",
5   "type": "module",
6   "scripts": {
7     "dev": "vite",
8     "build": "vite build",
9     "lint": "eslint .",
10    "preview": "vite preview"
11  },
12  "dependencies": {
13    "axios": "^1.13.5",
14    "clsx": "^2.1.1",
15    "framer-motion": "^12.34.1",
16    "i18next": "^25.8.18",
17    "i18next-browser-languagedetector": "^8.2.1",
18    "lucide-react": "^0.574.0",
19    "react": "^19.2.0",
20    "react-dom": "^19.2.0",
21    "react-i18next": "^16.5.8",
22    "react-router-dom": "^7.13.0",
23    "tailwind-merge": "^3.4.1"
24  },
25  "devDependencies": {
26    "@eslint/js": "^9.39.1",
27    "@types/react": "^19.2.7",
28    "@types/react-dom": "^19.2.3",
29    "@vitejs/plugin-react": "^5.1.1",
30    "autoprefixer": "^10.4.24",
31    "eslint": "^9.39.1",
32    "eslint-plugin-react-hooks": "^7.0.1",
33    "eslint-plugin-react-refresh": "^0.4.24",
34    "globals": "^16.5.0",
35    "postcss": "^8.5.6",
```

Extraits de code

Exemple du contrôleur d'authentification (AuthController.php) :

```
class AuthController extends Controller
{
    public function register(Request $request)
    {
        $validated = $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|string|email|max:255|unique:users',
            'password' => 'required|string|min:8|confirmed',
            'role' => 'required|in:user,artisan',
            'phone' => 'required|string|max:20',
            // Artisan specific fields
            'profession' => 'required_if:role,artisan|string|nullable',
        ]);

        $user = User::create([
            'name' => $validated['name'],
            'email' => $validated['email'],
            'password' => Hash::make($validated['password']),
            'role' => $validated['role'],
            'phone' => $validated['phone'],
        ]);

        if ($validated['role'] === 'artisan') {
            ArtisanProfile::create([
                'user_id' => $user->id,
                'profession' => $validated['profession'],
                'city' => 'Tadla', // Default city
                'available' => true,
            ]);
        }

        $token = $user->createToken('auth_token')->plainTextToken;

        return response()->json([
            'message' => 'User registered successfully',
            'user' => $user,
            'token' => $token,
        ], 201);
    }

    public function login(Request $request)
    {
        $validated = $request->validate([
            'email' => 'required|email',
            'password' => 'required',
        ]);

        $user = User::where('email', $validated['email'])->first();

        if (!$user || !Hash::check($validated['password'], $user->password)) {
            throw ValidationException::withMessages([
                'email' => ['Invalid credentials'],
            ]);
        }

        $token = $user->createToken('auth_token')->plainTextToken;

        return response()->json([
            'message' => 'Login successful',
            'user' => $user,
            'token' => $token,
            'role' => $user->role,
        ]);
    }

    public function logout(Request $request)
    {
        $request->user()->currentAccessToken()->delete();
        return response()->json(['message' => 'Logged out successfully']);
    }
}
```

Requêtes API du Backend

```

10 Route::post('/register', [AuthController::class, 'register']);
11 Route::post('/login', [AuthController::class, 'login']);
12 Route::post('/forgot-password', [AuthController::class, 'forgotPassword']);
13 Route::post('/reset-password', [AuthController::class, 'resetPassword']);
14
15 // Public routes
16 Route::get('/artisans', [ArtisanController::class, 'index']);
17 Route::get('/artisans/{id}/reviews', [ArtisanController::class, 'reviews']);
18 Route::get('/artisans/{id}', [ArtisanController::class, 'show']);
19 Route::get('/categories', [ArtisanController::class, 'categories']);
20
21 Route::middleware(['auth:sanctum'])->group(function () {
22
23     Route::post('/logout', [AuthController::class, 'logout']);
24
25     Route::get('/user', function (Request $request) {
26         return $request->user();
27     });
28
29     Route::get('/users/{id}', function ($id) {
30         $u = \App\Models\User::find($id);
31         if (!$u) {
32             return response()->json(['message' => 'Not found'], 404);
33         }
34
35         return response()->json([
36             'id' => $u->id,
37             'name' => $u->name,
38             'phone' => $u->phone,
39             'email' => $u->email
40         ]);
41     });
42
43     // Messaging
44     Route::get('/conversations', [MessageController::class, 'conversations']);
45     Route::get('/conversations/{userId}/messages', [MessageController::class, 'messages']);
46     Route::post('/messages', [MessageController::class, 'send']);
47
48     // Artisan Routes
49     Route::get('/artisan/profile', [ArtisanController::class, 'getProfile']);
50     Route::post('/artisan/profile', [ArtisanController::class, 'updateProfile']);
51     Route::post('/artisan/services', [ArtisanController::class, 'addService']);
52     Route::delete('/artisan/services/{service}', [ArtisanController::class, 'deleteService']);
53
54     Route::post('/artisan/photos/upload', [ArtisanController::class, 'uploadPhoto']);
55     Route::post('/artisan/photos', [ArtisanController::class, 'addPhoto']);
56     Route::delete('/artisan/photos/{photoId}', [ArtisanController::class, 'deletePhoto']);
57
58     Route::post('/artisans/{id}/reviews', [ArtisanController::class, 'storeReview']);
59     Route::get('/me/artisan-likes', [ArtisanController::class, 'likedArtisans']);
60     Route::post('/artisans/{id}/like', [ArtisanController::class, 'like']);
61     Route::delete('/artisans/{id}/like', [ArtisanController::class, 'unlike']);
62
63     // Admin Routes
64     Route::middleware('admin')->prefix('admin')->group(function () {
65         Route::get('/stats', [AdminController::class, 'stats']);
66         Route::get('/users', [AdminController::class, 'users']);
67         Route::get('/artisans', [AdminController::class, 'artisans']);
68         Route::put('/users/{user}', [AdminController::class, 'updateUser']);
69         Route::delete('/users/{user}', [AdminController::class, 'deleteUser']);
70     });
71
72 });
73
74 Route::get('/test', function () {
75     return response()->json([
76         'message' => 'Connected to Artisan Platform Backend'
77     ]);
78 });
79

```

Requêtes API du Frontend

```
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:8000/api',
  headers: {
    Accept: 'application/json',
  },
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');

  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }

  return config;
});

export const authService = {
  register: (data) => api.post('/register', data),
  login: (data) => api.post('/login', data),
  logout: () => api.post('/logout'),
  forgotPassword: (data) => api.post('/forgot-password', data),
  resetPassword: (data) => api.post('/reset-password', data),
};

export const artisanService = {
  getAll: (params) => api.get('/artisans', { params }),
  getOne: (id) => api.get(`/artisans/${id}`),
  getProfile: () => api.get('/artisan/profile'),
  updateProfile: (data) => api.post('/artisan/profile', data),
  addService: (data) => api.post('/artisan/services', data),
  deleteService: (id) => api.delete(`/artisan/services/${id}`),
  addPhoto: (data) => api.post('/artisan/photos', data),
  deletePhoto: (id) => api.delete(`/artisan/photos/${id}`),
  addReview: (id, data) => api.post(`/artisans/${id}/reviews`, data),
  getReviews: (id) => api.get(`/artisans/${id}/reviews`),
  getLiked: () => api.get('/me/artisan-likes'),
  like: (id) => api.post(`/artisans/${id}/like`),
  unlike: (id) => api.delete(`/artisans/${id}/like`),
};

export const messageService = {
  conversations: () => api.get('/conversations'),
  messages: (userId) => api.get(`/conversations/${userId}/messages`),
  send: (data) => api.post('/messages', data),
  getUser: (id) => api.get(`/users/${id}`),
};

export const adminService = {
  stats: () => api.get('/admin/stats'),
  users: (params) => api.get('/admin/users', { params }),
  artisans: (params) => api.get('/admin/artisans', { params }),
  updateUser: (id, data) => api.put(`/admin/users/${id}`, data),
  deleteUser: (id) => api.delete(`/admin/users/${id}`),
};

export default api;
```